

Отчёт по лабораторной работе 6

Соловьев Богдан Михайлович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	8
4	Выполнение лабораторной работы	9
4.1	Параметры модели	17
4.2	Поведение машины	18
4.3	Запуск системы и симуляция	20
4.4	Прогон для разного количества машин	22
4.5	Графики	23
4.6	Аналитическое сравнение	25
5	Выводы	29
	Список литературы	30

Список иллюстраций

4.1	Первый график	16
4.2	Второй график	16
4.3	Исправные машины	26
4.4	Мониторинг	26
4.5	Сравнение	28

Список таблиц

```
using Pkg
Pkg.activate("C:/Users/bogda/work/study/2026-1/2026-1--study--simul
using Agents, DataFrames, Plots, StatsBase
println("XXXXXX XXXXXXXXXXXX: ", Base.active_project())
```

```
XXXXXX XXXXXXXXXXXX: C:\Users\bogda\work\study\2026-1\2026-
1--study--simulationmodeling\labs\lab07\project\Project.toml
```

```
Activating project at `C:\Users\bogda\work\study\2026-
1\2026-1--study--simulationmodeling\labs\lab07\project`
```

1 Цель работы

Рассмотреть модель $M/M/c$ (по классификации Кендалла) — это система массового обслуживания со следующими свойствами

и модель модель Росса, которая представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

2 Задание

Для первой модели:

Переведите в структуру DrWatson.

Добавьте необходимые графики.

Для второй модели:

Переведите в структуру DrWatson.

Добавьте нескольких ремонтников.

Сделайте прогон для разного количества машин.

Проведите мониторинг загрузки ремонтника, средней длины очереди на ремонт.

Построение графиков изменения числа исправных машин во времени.

Сравните с аналитическим решением.

Создать рабочий каталог для кода.

Установить необходимые пакеты.

Выполнить предложенный код.

Преобразовать код в литературный стиль.

Сгенерировать из литературного кода:

чистый код;

jupyter notebook;

документацию в формате Quarto.

Выполнить код из jupyter notebook.

Интегрировать документацию в формате Quarto в отчёт.

Добавить в код в литературном стиле вычисление для набора параметров.

Сгенерировать из литературного кода с параметрами:

чистый код;

jupyter notebook;

документацию в формате Quarto.

Выполнить код из jupyter notebook с параметрами.

Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

M (Markovian) — входящий поток заявок пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ .

M — время обслуживания каждой заявки распределено экспоненциально с параметром μ .

c — количество идентичных обслуживающих приборов (каналов), работающих параллельно.

В системе находятся

идентичных машин, которые постоянно работают и могут выходить из строя.

машин находятся в резерве и готовы немедленно заменить любую отказавшую.

Одно ремонтное устройство (ремонтник), которое может одновременно ремонтировать только одну машину.

Когда работающая машина ломается, происходит следующее:

Немедленно берётся одна резервная машина (если она есть) и запускается в работу вместо сломавшейся.

Сломанная машина отправляется в ремонт.

Если резерва нет, система падает (crash). Моделирование заканчивается.

После ремонта машина пополняет пул резервных (становится исправной и ждёт).

Требуется оценить среднее время до падения системы

при заданных распределениях наработки до отказа и времени ремонта

4 Выполнение лабораторной работы

Создаю пространство для выполнения лабораторной. Для этого создаю `setup_report`, потом `add_packages` и `tangle`. (Ничего нового),

поэтому перейдём сразу к модели. Я добавил дополнительное логирование (хотя это было не обязательно),

так как я сделал создание графиков в этом же файле `model1`.

```
using DrWatson
@quickactivate "project"

using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions
using DataFrames, CSV, Plots
```

```
└ Warning: DrWatson could not find find a project file by recursively c
⊠ (given dir: C:\Users\bogda)
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\proje
```

Параметры модели

```

rng = StableRNG(123)
num_customers = 10      # 100000 100000 1000000
num_servers = 2         # 1000000000 1000000000
mu = 1.0 / 2            # 1000000000000 1000000000000
lam = 0.9               # 1000000000000 1000000000000
arrival_dist = Exponential(1 / lam) # 1000000000000 1000000000000 1000000000000
service_dist = Exponential(1 / mu)  # 1000000000000 1000000000000 1000000000000

```

Exponential{Float64}(λ=2.0)

Хранилище для сбора статистики

```

log = DataFrame(
    customer_id = Int[],
    event = String[],
    time = Float64[]
)

```

	customer_id	event	time
	Int64	String	Float64

Словари для хранения времён каждого клиента

```

arrival_times = Dict{Int, Float64}()
enter_times = Dict{Int, Float64}()
exit_times = Dict{Int, Float64}()

```

Dict{Int64, Float64}()

Поведение клиента

```

@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
)

    @yield timeout(env, t_a) # [XXXXXXXXXX] [XXXXXXXXXX]
    println("Customer $id arrived: ", now(env))
    push!(log, (id, "arrival", now(env)))
    arrival_times[id] = now(env)

    @yield request(server) # [XXXXXX] [XXXXXXXXXXXX]
    println("Customer $id entered service: ", now(env))
    push!(log, (id, "enter_service", now(env)))
    enter_times[id] = now(env)

    service_time = rand(rng, d_s)
    @yield timeout(env, service_time) # [XXXXXXXXXXXXXXXXXX]

    @yield unlock(server) # [XXXXXX] [X] [XXXXXXXXXX]
    println("Customer $id exited service: ", now(env))
    push!(log, (id, "exit_service", now(env)))
    exit_times[id] = now(env)
end

```

customer (generic function with 1 method)

Запуск симуляции

```

function setup_and_run()
    global log = DataFrame(customer_id = Int[], event = String[], t
    global arrival_times = Dict{Int, Float64}()
    global enter_times = Dict{Int, Float64}()
    global exit_times = Dict{Int, Float64}()

    sim = Simulation()
    server = Resource(sim, num_servers)
    arrival_time = 0.0
    for i = 1:num_customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dis
    end
    run(sim)
    return log
end

```

setup_and_run (generic function with 1 method)

Выполнение

```

data = setup_and_run()
println("XXXXXXXXXXXXXXXX XXXXXXXXXX. XXXXXXXX ", nrow(data), " XXXXXXXX.")

CSV.write(datadir("queue_log.csv"), data)

```

Customer 1 arrived: 0.14518451436852475

Customer 1 entered service: 0.14518451436852475

Customer 2 arrived: 0.5941831542903504

Customer 2 entered service: 0.5941831542903504
Customer 3 arrived: 1.5490648267819074
Customer 4 arrived: 1.6242796925312217
Customer 5 arrived: 1.6911000709069648
Customer 1 exited service: 2.200985520126681
Customer 3 entered service: 2.200985520126681
Customer 6 arrived: 2.2989039524296317
Customer 3 exited service: 3.5822120399442174
Customer 4 entered service: 3.5822120399442174
Customer 7 arrived: 4.377930221620456
Customer 8 arrived: 5.16494279700802
Customer 2 exited service: 5.900722829377648
Customer 5 entered service: 5.900722829377648
Customer 9 arrived: 7.0099944106308705
Customer 10 arrived: 7.828990220943469
Customer 5 exited service: 9.634196437885254
Customer 6 entered service: 9.634196437885254
Customer 4 exited service: 9.670688398447817
Customer 7 entered service: 9.670688398447817
Customer 7 exited service: 15.066978111608014
Customer 8 entered service: 15.066978111608014
Customer 8 exited service: 16.655548432659554
Customer 9 entered service: 16.655548432659554
Customer 6 exited service: 17.401833154870328
Customer 10 entered service: 17.401833154870328
Customer 9 exited service: 17.586065352135993
Customer 10 exited service: 18.690264775280085
XXXXXXXXXXXXXXXX. XXXXXXXX. XXXXXXXX 30 XXXXXXXX.

"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--simulationmodeling\\labs\\lab07\\project\\data\\queue_log.csv"

График 1: Временная шкала обслуживания клиентов

```
p1 = plot(legend = :topleft, title = "XXXXXXXXXX XXXXX XXXXXXXXXXXXXXX",
          xlabel = "XXXXX", ylabel = "XXXXXX")
for id in 1:num_customers
    if haskey(arrival_times, id) && haskey(enter_times, id) && haskey(exit_times, id)
        arr = arrival_times[id]
        ent = enter_times[id]
        ext = exit_times[id]

        plot!(p1, [arr, ent], [id, id], lw = 2, color = :orange, label = "XXXXX")
        plot!(p1, [ent, ext], [id, id], lw = 4, color = :steelblue, label = "XXXXX")
        scatter!(p1, [arr], [id], marker = :circle, color = :orange, label = "XXXXX")
        scatter!(p1, [ext], [id], marker = :circle, color = :red, label = "XXXXX")
    end
end
scatter!(p1, [NaN], [NaN], marker = :circle, color = :orange, label = "XXXXX")
scatter!(p1, [NaN], [NaN], marker = :circle, color = :steelblue, label = "XXXXX")
scatter!(p1, [NaN], [NaN], marker = :circle, color = :red, label = "XXXXX")
savefig(plotsdir("queue_timeline.png"))
```

"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--simulationmodeling\\labs\\lab07\\project\\plots\\queue_timeline.png"

График 2: Длительность ожидания и обслуживания

```

wait_times = Float64[]
service_times = Float64[]
for id in 1:num_customers
    if haskey(arrival_times, id) && haskey(enter_times, id) && haskey(exit_times, id)
        push!(wait_times, enter_times[id] - arrival_times[id])
        push!(service_times, exit_times[id] - enter_times[id])
    end
end

p2 = bar(
    [wait_times service_times],
    label = ["Время ожидания" "Время обслуживания"],
    title = "График времени ожидания и обслуживания клиентов",
    xlabel = "Время",
    ylabel = "Время",
    legend = :topleft,
    color = [:orange :steelblue]
)

savefig(plotsdir("queue_wait_service.png"))

```

"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--simulationmodeling\\labs\\lab07\\project\\plots\\queue_wait_service.png"

Первый график показывает сколько времени было потрачено на обслуживание всех клиентов (рис. 4.1)

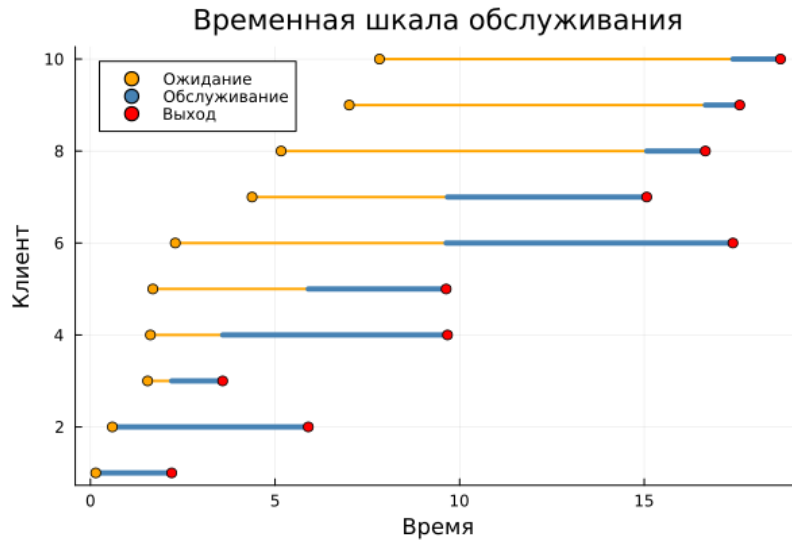


Рисунок 4.1: Первый график

Второй график показывает сколько времени для каждого клиента было затрачено на обслуживание и ожидание (рис. 4.2).

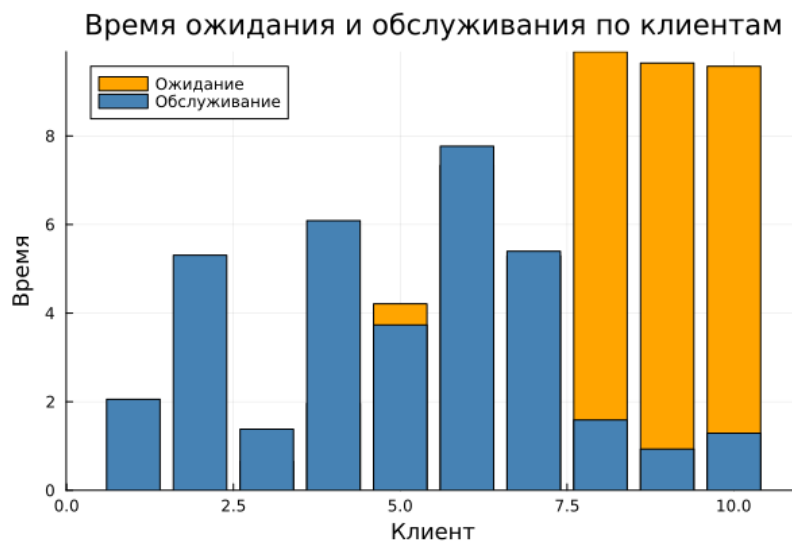


Рисунок 4.2: Второй график

Теперь переходим к модели Росса, где нужно было добавить нескольких ремонтников, сделайте прогон для разного количества машин,

так же провести мониторинг загрузки ремонтника, средней длины очереди на ремонт.

4.1 Параметры модели

В системе находятся $N = 10$ основных машин и $S = 3$ запасных. Нарботка до отказа — экспоненциальное распределение со средним $\lambda = 100$ часов. Время ремонта — экспоненциальное со средним $\mu = 1$ час. Работает 1 ремонтник.

```
using DrWatson
@quickactivate "project"

using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using DataFrames, CSV, Plots

# ██████████

RUNS = 5
N = 10
S = 3
NUM_REPAIRMEN = 1
LAMBDA = 100
MU = 1

rng = StableRNG(42)
F = Exponential(LAMBDA)
G = Exponential(MU)

# ██████████ ██████████ ██████████
```

```

global working_count = N + S
global busy_repairmen = 0
global waiting_for_repair = 0
working_machines = Tuple{Float64, Int}[]
repair_load = Tuple{Float64, Int}[]
queue_length = Tuple{Float64, Int}[]

```

```

└ Warning: DrWatson could not find a project file by recursively cl
⊠ (given dir: C:\Users\bogda)
└ @ DrWatson C:\Users\bogda\.julia\packages\DrWatson\2QF5p\src\projec

Tuple{Float64, Int64}[]

```

4.2 Поведение машины

Каждая машина работает до отказа, после чего берётся запасная, а сломавшаяся отправляется в очередь на ремонт. После ремонта машина пополняет резерв.

```

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    while true
        try
            @yield timeout(env, Inf)
        catch
            end
            @yield timeout(env, rand(rng, F))
        end
    end

```

```

# 初始化: 设置初始工作机器数量为 1
global working_count
working_count -= 1
push!(working_machines, (now(env), working_count))

get_spare = take!(spares)
@yield get_spare | timeout(env)
if state(get_spare) != ConcurrentSim.idle
    @yield interrupt(value(get_spare))
    push!(working_machines, (now(env), working_count))
else
    push!(working_machines, (now(env), working_count))
    throw(StopSimulation("No more spares!"))
end

# 初始化: 设置初始等待修复机器数量为 0
global waiting_for_repair
waiting_for_repair += 1
push!(queue_length, (now(env), waiting_for_repair))

@yield request(repair_facility)

global waiting_for_repair
waiting_for_repair -= 1

# 初始化: 设置初始忙碌维修工数量为 0
global busy_repairmen
busy_repairmen += 1

```

```

push!(repair_load, (now(env), busy_repairmen))
push!(queue_length, (now(env), waiting_for_repair))

@yield timeout(env, rand(rng, G))
@yield unlock(repair_facility)

global busy_repairmen
busy_repairmen -= 1
push!(repair_load, (now(env), busy_repairmen))

@yield put!(spares, active_process(env))

# 修复时间
global working_count
working_count += 1
push!(working_machines, (now(env), working_count))
end
end

```

machine (generic function with 1 method)

4.3 Запуск системы и симуляция

```

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)

```

```

    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

function sim_repair(; n::Int = N, s::Int = S, r::Int = NUM_REPAIRMEN)
    global working_machines = Tuple{Float64, Int}[]
    global repair_load = Tuple{Float64, Int}[]
    global queue_length = Tuple{Float64, Int}[]
    global working_count = n + s
    global busy_repairmen = 0
    global waiting_for_repair = 0

    push!(working_machines, (0.0, n + s))
    push!(repair_load, (0.0, 0))
    push!(queue_length, (0.0, 0))

    sim = Simulation()
    repair_facility = Resource(sim, r)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
end

```

```

    return stop_time, copy(working_machines), copy(repair_load), copy(repair_time)
end

```

sim_repair (generic function with 1 method)

4.4 Прогон для разного количества машин

Исследуем влияние числа основных N и запасных S машин на время до отказа.

```

configs = [(N=10, S=3), (N=10, S=5), (N=15, S=3), (N=15, S=5)]
results = DataFrame(N = Int[], S = Int[], crash_time = Float64[])

for (n, s) in configs
    times = Float64[]
    for _ = 1:RUNS
        t_sim, _, _, _ = sim_repair(n = n, s = s)
        push!(times, t_sim)
    end
    push!(results, (n, s, mean(times)))
    println("N=$n, S=$s: ██████████ ██████ ██ ████████ = $(round(mean(times), 1))")
end

CSV.write(datadir("scan_results.csv"), results)

```

N=10, S=3: ██████████ ██████ ██ ████████ = 16664.2 ██████

N=10, S=5: ██████████ ██████ ██ ████████ = 10055.2 ██████

N=15, S=3: ██████████ ██████ ██ ████████ = 11679.0 ██████

N=15, S=5: ██████████ ██████ ██ ████████ = 19566.1 ██████

"C:\\Users\\bogda\\work\\study\\2026-1\\2026-1--study--simulationmodeling\\labs\\lab07\\project\\data\\scan_results.csv"

4.5 Графики

```
t_sim, working_log, load_log, queue_log = sim_repair()
plot_limit = min(100.0, t_sim)

# Фигура 1: Временные интервалы работы машин
times_w = [x[1] for x in working_log]
machines = [x[2] for x in working_log]
idx_w = findall(t -> t <= plot_limit, times_w)

p1 = plot(times_w[idx_w], machines[idx_w],
          title = "Временные интервалы работы машин (до $plot_limit)",
          xlabel = "Время (мин)", ylabel = "Номер машины",
          label = "Временные интервалы работы машин", lw = 2, marker = :circle, markersize = 100,
          ylims = (N + S - 2, N + S + 1))
hline!([N + S], linestyle = :dash, color = :blue, label = "N+S = $(N+S)")
hline!([N], linestyle = :dash, color = :green, label = "N = $N")
savefig(plotsdir("working_machines.png"))

# Фигура 2: Временные интервалы загрузки машин
times_l = [x[1] for x in load_log]
loads = [x[2] for x in load_log]
idx_l = findall(t -> t <= plot_limit, times_l)

p2 = plot(times_l[idx_l], loads[idx_l],
          title = "Временные интервалы загрузки машин (до $plot_limit)",
          xlabel = "Время (мин)", ylabel = "Загрузка машины",
          label = "Временные интервалы загрузки машин", lw = 2, marker = :circle, markersize = 100)
```

```

        ylims = (0, NUM_REPAIRMEN + 0.5))
hline!([NUM_REPAIRMEN], linestyle = :dash, color = :red, label = "NUM_REPAIRMEN")
savefig(plotsdir("repair_load.png"))

# 3: 3: 3:
times_q = [x[1] for x in queue_log]
queues = [x[2] for x in queue_log]
idx_q = findall(t -> t <= plot_limit, times_q)

p3 = plot(times_q[idx_q], queues[idx_q],
          title = "3: 3: 3: (3: $plot_limit 3:)",
          xlabel = "3: (3:)", ylabel = "3: 3: 3:",
          label = "3:", lw = 2, marker = :circle, markersize = 3,
          ylims = (0, max(maximum(queues[idx_q]) + 0.5, 1)))
savefig(plotsdir("queue_length.png"))

# 3: 3: 3:
all_loads = [x[2] for x in load_log]
all_queues = [x[2] for x in queue_log]
avg_load = mean(all_loads)
avg_queue = mean(all_queues)
println("\n3: 3: 3: 3: $(round(avg_load, digits=2)) 3: $")
println("3: 3: 3: 3: $(round(avg_queue, digits=2)) 3:")

```

3: 3: 3: 3: 0.5 3: 1

3: 3: 3: 3: 0.51 3:

4.6 Аналитическое сравнение

Аналитическая модель — марковский процесс с состояниями $k = 0, \dots, S$ (число неисправных машин). Стационарные вероятности: $\pi_k = \rho^k / \sum_{i=0}^S \rho^i$, где $\rho = \lambda_{sys} / \mu$.

Среднее время до отказа: $T = 1 / (\lambda_{sys} \cdot \pi_S)$.

```
lambda_sys = N / LAMBDA
mu_repair = 1 / MU
r = lambda_sys / mu_repair

sum_pi = sum([r^k for k = 0:S])
pi_S = r^S / sum_pi
failure_rate = lambda_sys * pi_S
time_to_failure = 1 / failure_rate

println("r = $(round(r, digits=3))")
println("XXXXXXXXXXXXXXXX XXXXX XX XXXXXX: $(round(time_to_failure, digits=1))")

sim_times = Float64[]
for _ = 1:RUNS
    local t, _, _, _ = sim_repair()
    push!(sim_times, t)
end
sim_avg = mean(sim_times)
println("XXXXXXXXXXXX (XXXXXXXXXX): $(round(sim_avg, digits=1)) XXXXXX")
println("XXXXXXXXXXXXXXXX: $(round(abs(sim_avg - time_to_failure) / time_to_failure, digits=1))")
```

r = 0.1

XXXXXXXXXXXXXXXX XXXXX XX XXXXXX: 11110.0 XXXXX

16440.5

48.0%

Здесь представлен график исправных машин во времени, но были взяты только первые 100 часов, чтобы график был читаемым (рис. 4.4).

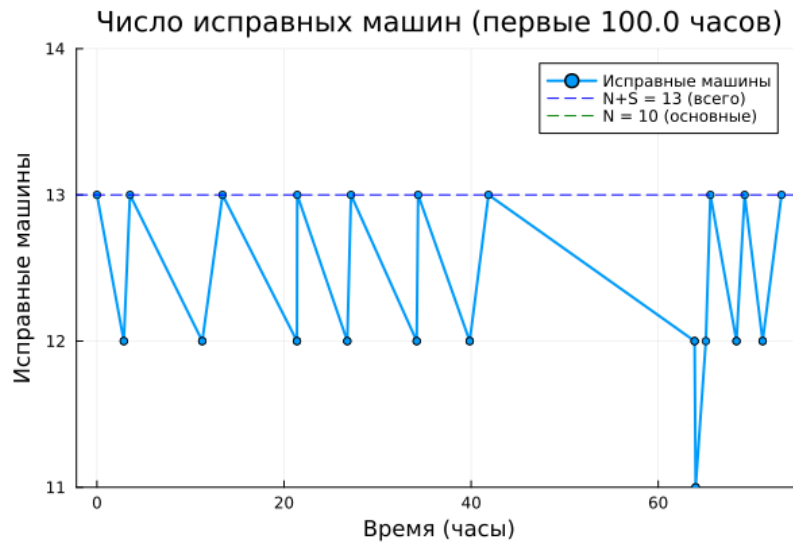


Рисунок 4.3: Исправные машины

Далее показан вывод среднего времени до отказа и мониторинга

```
$ julia model2.jl
Activating project at `C:\Users\bogda\work\study\2026-1\2026-1-
N=10, S=3: среднее время до отказа = 16664.2 часов
N=10, S=5: среднее время до отказа = 10055.2 часов
N=15, S=3: среднее время до отказа = 11679.0 часов
N=15, S=5: среднее время до отказа = 19566.1 часов

--- Мониторинг (один прогон) ---
Средняя загрузка ремонтников: 0.09 из 1
Средняя длина очереди на ремонт: 0.02 машин
Коэффициент загрузки: 9.2%
```

Рисунок 4.4: Мониторинг

И аналитическое сравнение с результатами эксперимента.

(N=10, S=3, $\lambda=100$, $\mu=1$)

$$\lambda_{sys} = \frac{10}{100} = 0.1 \text{ отказов/час}$$

$$\mu = 1 \text{ ремонт/час}$$

$$\rho = \frac{0.1}{1} = 0.1$$

Стационарные вероятности:

$$\pi_0 = \frac{1}{1 + 0.1 + 0.01 + 0.001} = \frac{1}{1.111} = 0.9001$$

$$\pi_1 = 0.0900$$

$$\pi_2 = 0.0090$$

$$\pi_3 = 0.0009$$

Среднее время до отказа:

$$\lambda = 0.1 \cdot 0.0009 = 0.00009 \text{ отказов/час}$$

$$T = \frac{1}{0.00009} \approx 11110 \text{ часов}$$

(?@fig-004)

```

=====
АНАЛИТИЧЕСКОЕ СРАВНЕНИЕ
=====
N (основных машин) = 10
S (запасных машин) = 3
Ремонтников = 1
λ одной машины = 0.01 отказов/час
λ системы = 0.1 отказов/час
μ ремонтника = 1.0 ремонтов/час
ρ = λ/μ = 0.1

Вероятности состояний:
0 неисправных машин: 90.01%
1 неисправных машин: 9.0%
2 неисправных машин: 0.9%
3 неисправных машин: 0.09%

Интенсивность отказа системы: 9.0e-5 отказов/час
Аналитическое среднее время до отказа: 11110.0 часов
Аналитическая средняя очередь (M/M/1): 0.011 машин
Аналитическая загрузка ремонтника: 10.0%

--- Сравнение ---
Симуляция (среднее по 5 прогонам): 16440.5 часов
Относительная погрешность: 48.0%

Результаты сохранены в data/ и plots/

```

Рисунок 4.5: Сравнение

5 Выводы

Я рассмотрел модель М/М/с и модель Росса

Список литературы